

HTML templates

HTML templates

For a single HTML notebook, use `layouts/notebook.html`. For websites, the default entry is `layouts/webpage.html`.

Layouts are MiniJinja templates. The template context contains these top-level values:

- `doc`: Typst's generated HTML shell and document content.
- `site`: website metadata, navigation, and search settings.
- `css`: theme CSS files. Each item has `name` and `content`.
- `js`: theme JS files. Each item has `name` and `content`.
- `vars`: custom values from `[vars]` in `calepin.toml`.
- `highlight_css`: standalone syntax-highlight CSS.
- `theme`: the active theme name.
- `target`: the render target; currently `html`.

`doc` contains:

- `doc.head`: Typst's document shell before `</head>`.
- `doc.body_open`: `</head>` and Typst's opening `<body>` tag.
- `doc.body`: Typst's generated body content.
- `doc.body_close`: Typst's closing `</body>` tag and any remaining document shell.
- `doc.title`: the document title, or an empty string.

`site` contains:

- `site.title`, `site.description`, `site.base_url`
- `site.logo`, `site.logo_alt`, `site.home_url`, `site.favicon`
- `site.page_url`, `site.page_title`
- `site.sidebar`: flat navigation entries.
- `site.sidebar_sections`: grouped navigation sections.
- `site.sidebar_fold`: whether titled sidebar sections should fold.
- `site.toc`: the current page table of contents.
- `site.menus`: named menus, such as `site.menus.main`, `site.menus.social`, and `site.menus.footer`.
- `site.language`: the current language code.
- `site.languages`: language entries for the language picker.
- `site.translations`: alternate-language links for the current page.
- `site.pagefind`: Pagefind search assets and bundle path, when search is enabled.

Nested entries expose these fields:

- Navigation entries in `site.sidebar`, `site.menus.<name>`, and section items: `href`, `label`, `label_html`, `active`.
- Sidebar sections: `title`, `active`, `items`.
- TOC entries: `level`, `href`, `label`.
- Language and translation entries: `code`, `label`, `href`, `active`.
- Pagefind: `css`, `js`, `bundle`.

Minimal template

Here is a minimal `layouts/notebook.html`:

```

{{ doc.head }}
<meta charset="UTF-8">
<title>{{ doc.title }}</title>
{% for file in css %}
<style>
{{ file.content }}
</style>
{% endfor %}
{{ doc.body_open }}
<header class="document-header">
<a href="index.html">Home</a>
<button type="button" data-calepin-theme-toggle>Theme</button>
</header>
<main class="container">
{{ doc.body }}
</main>
{% for file in js %}
<script>
{{ file.content }}
</script>
{% endfor %}
{{ doc.body_close }}

```

Keep `doc.head`, `doc.body_open`, and `doc.body_close` unless you are intentionally replacing the entire HTML shell.

Custom variables

Add a `[vars]` table to `calepin.toml` for project-specific values you want to use in templates:

```

[vars]
course = "Econ 101"
semester = "Fall 2026"

```

These values are available as top-level vars, not under `site`:

```

<p>{{ vars.course }} - {{ vars.semester }}</p>

```

Partials

Keep repeated HTML in partials under `partials/` and include them from layouts.

```

{% include "partials/header.html" %}

```

Partials receive the same template context as the file that includes them.

Page-specific layouts

Sometimes, we would like a specific page of a website to use a different layout than the default. For instance, the landing page is often very different than the other pages of a site.

You can switch layouts per page with `<website-metadata>`:

```

#metadata((
  title: "Landing page",
  layout: "layouts/landing.html",
)) <website-metadata>

```

The `layout` value must be a relative `.html` path inside the active theme. Calepin does not add `layouts/` or `.html` for you and does not fall back to `layouts/webpage.html` if the file is missing.